

Praktikum 3: Threads & Concurrency

Informasi Praktikum

- **Mata Kuliah:** Sistem Operasi
 - **Sistem Operasi:** Linux Ubuntu
 - **Bahasa Pemrograman:** C
 - **Dosen Pengampu:** Triawan Adi Cahyanto, M.Kom
 - **Asisten Dosen:** Atidhira Habibillah dan Taqiyyuddin
-

Topik 1: Membuat Thread Sederhana

Tujuan

Memahami cara membuat dan menjalankan thread menggunakan POSIX Threads (pthread) di Linux.

Penjelasan Konsep

Thread adalah "pekerja kecil" dalam program Anda. Bayangkan Anda memiliki pabrik (program), dan thread adalah pekerja yang bisa mengerjakan tugas berbeda secara bersamaan.

Langkah-langkah

1. Buat file program baru:

```
nano thread_sederhana.c
```

2. Ketik kode berikut:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Fungsi yang akan dijalankan oleh thread
void* cetak_pesan(void* arg) {
    char* pesan = (char*) arg;

    for(int i = 1; i <= 5; i++) {
        printf("%s - hitungan ke-%d\n", pesan, i);
        sleep(1); // Tidur 1 detik
    }

    return NULL;
}

int main() {
```

```
pthread_t thread1, thread2;

printf("=== Program Dimulai ===\n\n");

// Membuat 2 thread
pthread_create(&thread1, NULL, cetak_pesan, "Thread 1");
pthread_create(&thread2, NULL, cetak_pesan, "Thread 2");

// Menunggu kedua thread selesai
pthread_join(thread1, NULL);
pthread_join(thread2, NULL);

printf("\n=== Program Selesai ===\n");

return 0;
}
```

3. Compile program:

```
gcc thread_sederhana.c -o thread_sederhana -pthread
```

4. Jalankan program:

```
./thread_sederhana
```

Apa Yang Terjadi? ...

Topik 2: Thread dengan Shared Memory

Tujuan

Memahami bagaimana thread berbagi memori dan masalah yang bisa terjadi.

Penjelasan Konsep

Thread dalam satu proses berbagi memori yang sama. Ini seperti beberapa pekerja menggunakan papan tulis yang sama - mereka bisa saling menimpa tulisan!

Langkah-langkah

1. Buat file baru:

```
nano thread_shared.c
```

2. Ketik kode berikut:

```
#include <stdio.h>
#include <pthread.h>

int counter = 0; // Variabel yang dibagi bersama

void* tambah_counter(void* arg) {
    for(int i = 0; i < 100000; i++) {
        counter++; // Menambah counter
    }
    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    printf("Counter awal: %d\n", counter);

    // Buat 2 thread yang menambah counter
    pthread_create(&thread1, NULL, tambah_counter, NULL);
    pthread_create(&thread2, NULL, tambah_counter, NULL);

    // Tunggu kedua thread selesai
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Counter akhir: %d\n", counter);
    printf("Harusnya: 200000\n");

    return 0;
}
```

3. Compile dan jalankan:

```
gcc thread_shared.c -o thread_shared -pthread
./thread_shared
```

Apa Yang Terjadi? ...

Topik 3: Mengatasi Race Condition dengan Mutex

Tujuan

Belajar menggunakan mutex (mutual exclusion) untuk mengamankan shared data.

Penjelasan Konsep

Mutex seperti "kunci pintu". Hanya satu thread yang boleh masuk ke area kritis (critical section) pada satu waktu.

Langkah-langkah

1. Buat file baru:

```
nano thread_mutex.c
```

2. Ketik kode berikut:

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock; // Variabel mutex (kunci)

void* tambah_counter(void* arg) {
    for(int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);    // Kunci pintu
        counter++;                    // Area aman
        pthread_mutex_unlock(&lock);  // Buka kunci
    }
    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    // Inisialisasi mutex
    pthread_mutex_init(&lock, NULL);

    printf("Counter awal: %d\n", counter);

    pthread_create(&thread1, NULL, tambah_counter, NULL);
    pthread_create(&thread2, NULL, tambah_counter, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Counter akhir: %d\n", counter);
    printf("Harusnya: 200000\n");

    // Hapus mutex
    pthread_mutex_destroy(&lock);

    return 0;
}
```

3. Compile dan jalankan:

```
gcc thread_mutex.c -o thread_mutex -pthread
./thread_mutex
```

Apa Yang Terjadi? ...

Topik 4: Thread Pool Sederhana

Tujuan

Memahami konsep thread pool untuk efisiensi.

Penjelasan Konsep

Daripada membuat thread baru setiap kali ada tugas, lebih baik punya "pool pekerja" yang siap mengerjakan tugas kapan saja.

Langkah-langkah

1. Buat file baru:

```
nano thread_pool.c
```

2. Ketik kode berikut:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define JUMLAH_THREAD 3
#define JUMLAH_TUGAS 10

void* kerjakan_tugas(void* arg) {
    int id = *((int*) arg);

    printf("Thread %d: Mulai mengerjakan tugas\n", id);
    sleep(2); // Simulasi pekerjaan
    printf("Thread %d: Selesai mengerjakan tugas\n", id);

    return NULL;
}

int main() {
    pthread_t threads[JUMLAH_THREAD];
    int thread_ids[JUMLAH_THREAD];
```

```
printf("=== Thread Pool dengan %d threads ===\n\n", JUMLAH_THREAD);

// Buat pool of threads
for(int i = 0; i < JUMLAH_THREAD; i++) {
    thread_ids[i] = i + 1;
    pthread_create(&threads[i], NULL, kerjakan_tugas, &thread_ids[i]);
}

// Tunggu semua thread selesai
for(int i = 0; i < JUMLAH_THREAD; i++) {
    pthread_join(threads[i], NULL);
}

printf("\n=== Semua tugas selesai ===\n");

return 0;
}
```

3. Compile dan jalankan:

```
gcc thread_pool.c -o thread_pool -pthread
./thread_pool
```

Apa Yang Terjadi? ...

Topik 5: Data Parallelism

Tujuan

Memahami pemrosesan data paralel menggunakan multiple threads.

Penjelasan Konsep

Membagi data besar menjadi bagian-bagian kecil dan memprosesnya secara paralel.

Langkah-langkah

1. Buat file baru:

```
nano data_parallel.c
```

2. Ketik kode berikut:

```
#include <stdio.h>
#include <pthread.h>
```

```

#define UKURAN_ARRAY 1000
#define JUMLAH_THREAD 4

int array[UKURAN_ARRAY];
int jumlah_parsial[JUMLAH_THREAD];

typedef struct {
    int id;
    int mulai;
    int akhir;
} ThreadData;

void* hitung_jumlah(void* arg) {
    ThreadData* data = (ThreadData*) arg;
    int sum = 0;

    for(int i = data->mulai; i < data->akhir; i++) {
        sum += array[i];
    }

    jumlah_parsial[data->id] = sum;
    printf("Thread %d: Menghitung indeks %d-%d, hasil = %d\n",
        data->id, data->mulai, data->akhir-1, sum);

    return NULL;
}

int main() {
    pthread_t threads[JUMLAH_THREAD];
    ThreadData thread_data[JUMLAH_THREAD];

    // Isi array dengan angka 1-1000
    for(int i = 0; i < UKURAN_ARRAY; i++) {
        array[i] = i + 1;
    }

    printf("=== Data Parallelism: Menjumlahkan array ===\n\n");

    int bagian = UKURAN_ARRAY / JUMLAH_THREAD;

    // Buat thread untuk memproses bagian array
    for(int i = 0; i < JUMLAH_THREAD; i++) {
        thread_data[i].id = i;
        thread_data[i].mulai = i * bagian;
        thread_data[i].akhir = (i + 1) * bagian;

        pthread_create(&threads[i], NULL, hitung_jumlah, &thread_data[i]);
    }

    // Tunggu semua thread selesai
    for(int i = 0; i < JUMLAH_THREAD; i++) {
        pthread_join(threads[i], NULL);
    }
}

```

```
// Jumlahkan hasil dari semua thread
int total = 0;
for(int i = 0; i < JUMLAH_THREAD; i++) {
    total += jumlah_parsial[i];
}

printf("\n=== Total keseluruhan: %d ===\n", total);
printf("Verifikasi: 1+2+...+1000 = %d\n", (UKURAN_ARRAY * (UKURAN_ARRAY + 1))
/ 2);

return 0;
}
```

3. Compile dan jalankan:

```
gcc data_parallel.c -o data_parallel -pthread
./data_parallel
```

Apa Yang Terjadi? ...

Topik 6: Mendeteksi Jumlah CPU Core

Tujuan

Belajar mengoptimalkan program berdasarkan jumlah core processor.

Langkah-langkah

1. Buat file baru:

```
nano cpu_cores.c
```

2. Ketik kode berikut:

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

void* tugas_berat(void* arg) {
    int id = *((int*) arg);
    long hasil = 0;

    printf("Thread %d: Memulai komputasi...\n", id);

    // Simulasi komputasi berat
    for(long i = 0; i < 100000000; i++) {
```

```
        hasil += i;
    }

    printf("Thread %d: Selesai!\n", id);
    return NULL;
}

int main() {
    // Deteksi jumlah CPU core
    int jumlah_core = sysconf(_SC_NPROCESSORS_ONLN);

    printf("=== Informasi Sistem ===\n");
    printf("Jumlah CPU core: %d\n\n", jumlah_core);

    pthread_t threads[jumlah_core];
    int thread_ids[jumlah_core];

    printf("Membuat %d thread (sesuai jumlah core)...\n\n", jumlah_core);

    // Buat thread sebanyak jumlah core
    for(int i = 0; i < jumlah_core; i++) {
        thread_ids[i] = i + 1;
        pthread_create(&threads[i], NULL, tugas_berat, &thread_ids[i]);
    }

    // Tunggu semua selesai
    for(int i = 0; i < jumlah_core; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("\n=== Semua thread selesai ===\n");

    return 0;
}
```

3. Compile dan jalankan:

```
gcc cpu_cores.c -o cpu_cores -pthread
./cpu_cores
```

Monitor Penggunaan CPU

Buka terminal baru dan jalankan:

```
htop
```

atau

[top](#)

Lihat bagaimana semua core CPU Anda bekerja!

Tugas Praktikum

Tugas 1: Kalkulator Paralel

Buat program yang:

1. Menerima input 2 angka dari user
2. Menghitung penjumlahan, pengurangan, perkalian, dan pembagian
3. Setiap operasi dikerjakan oleh thread berbeda
4. Tampilkan semua hasil setelah semua thread selesai

Tugas 2: File Processing

Buat program yang:

1. Membaca file teks
2. Menghitung jumlah kata menggunakan 2 thread (bagi file jadi 2 bagian)
3. Gabungkan hasilnya
4. Bandingkan waktu eksekusi dengan versi single-thread

Tugas 3: Producer-Consumer

Buat simulasi producer-consumer dengan:

1. Thread producer yang menghasilkan angka random
2. Thread consumer yang memproses angka tersebut
3. Gunakan mutex untuk sinkronisasi
4. Buffer maksimal 10 item

Disclaimer

Boleh pakai AI tapi tidak boleh pure copy paste, harus disusun ulang menggunakan kalimat Anda sendiri. Ketika terdeteksi tugas disusun menggunakan AI maka langsung diberi nilai lulus minimal. Yang dilaporkan pada tugas adalah kode program, luaran kode program (screenshot) dan penjelasan maksud dari kode program dan hasil analisis dari kode program tersebut. Apabila ada dokumentasi error, silakan ditambahkan juga dan beri keterangan.

Tips & Troubleshooting

Error: undefined reference to pthread_create

Solusi: Pastikan menggunakan flag `-pthread` saat compile

```
gcc program.c -o program -pthread
```

Program Hang/Tidak Selesai

Penyebab: Lupa `pthread_join()` atau deadlock **Solusi:** Pastikan setiap thread yang dibuat di-join

Hasil Tidak Konsisten

Penyebab: Race condition **Solusi:** Gunakan mutex untuk area kritis

Compile Error

Solusi: Pastikan header `<pthread.h>` sudah di-include

Perintah Berguna

Melihat thread yang berjalan

```
ps -elf | grep nama_program
```

Monitor performa

```
htop
```

Mengecek dokumentasi pthread

```
man pthread_create  
man pthread_join  
man pthread_mutex_init
```

Referensi & Bacaan Lanjutan

1. Manual pthread: `man pthreads`
2. Tutorial lengkap: <https://www.cs.cmu.edu/afs/cs/academic/class/15492-f07/www/pthreads.html>
3. POSIX Threads Programming: <https://computing.lln.gov/tutorials/pthreads/>